Universität Zürich UZH

GEO877

Spatial Algorithm

01. Distance

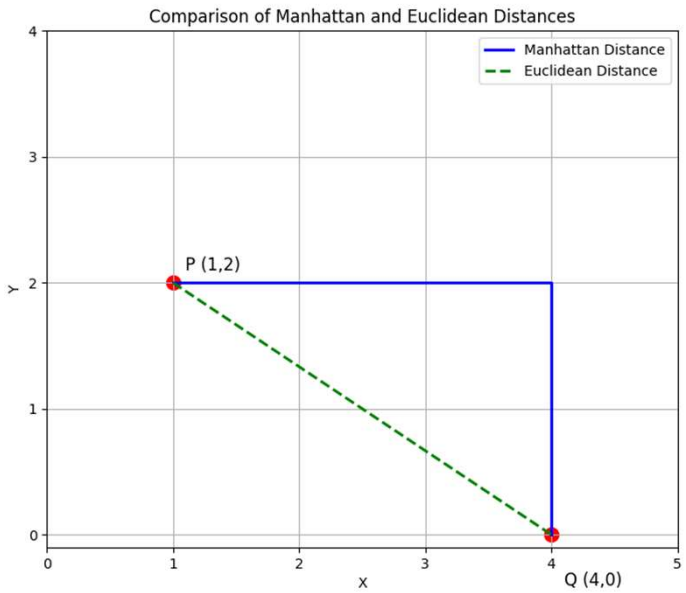
Inhye Kong

`Point` class

- The most basic geometric data: a single data point
- Property:
  - At spatial representation of 2-dimension, it is in a coordinate (x, y)
- Behaviour:
  - Equality: we can test if two (or more) points are the same
  - **Distance**: we can measure the spatial (2 dimensional) distance between two points

Distance

- Calculating distance is almost always, calculating the shortest distance
- In Cartesian coordinates, you can use **Euclidean** distance
- In semi-real-world, where the streets are grid-based and we cannot go through the building blocks, you can use **Manhattan** distance



https://www.geeksforgeeks.org/data-science/manhattan-distance/

GEO876 recap:

Finding the nearest point pair

https://www.arcgis.com/home/item.html?id=8868a283da884257bfc032c730a80685; original data comes from Open Data Zürich

GEO876 recap:

Finding the nearest point pair

Brute force approach

```
Dist = 99999 # big number for the context
for p in points
    for q in points
    if p is not q
        d = distance between p and q
        if d < Dist
            Dist = d
print(Dist)
```

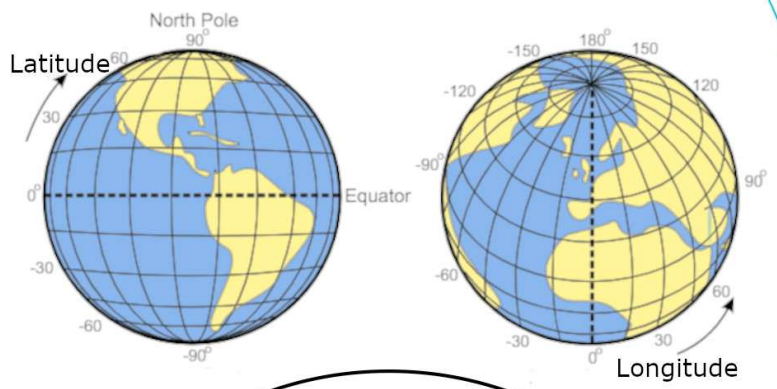
Is it correct? Any possibilities of not working as planned?
Is it efficient? How much is the computational load?

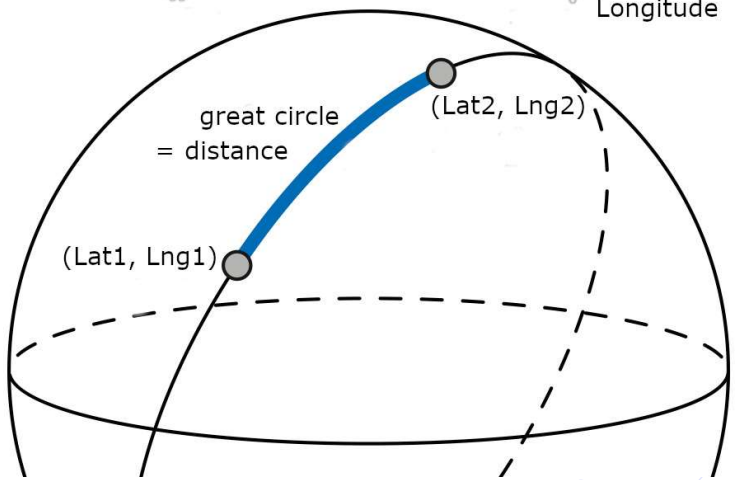
Efficiency = $O(n^2)$

To learn more about this "Big O notation", check out the link below
https://www.geeksforgeeks.org/dsa/analysis-algorithms-big-o-analysis/

Great-circle distance

- Cartesian distance does not always scale
- In a real-world situation, where the distance measure occurs at a large scale on earth, e.g. flying over the sphere, or when you measure very fine-grained distances
- **Haversine** distance, i.e. great circle distance, can be a solution
- Applications: GPS navigation, flight path planning, proximity search, mapping APIs (Google Maps, OpenStreetMap)…



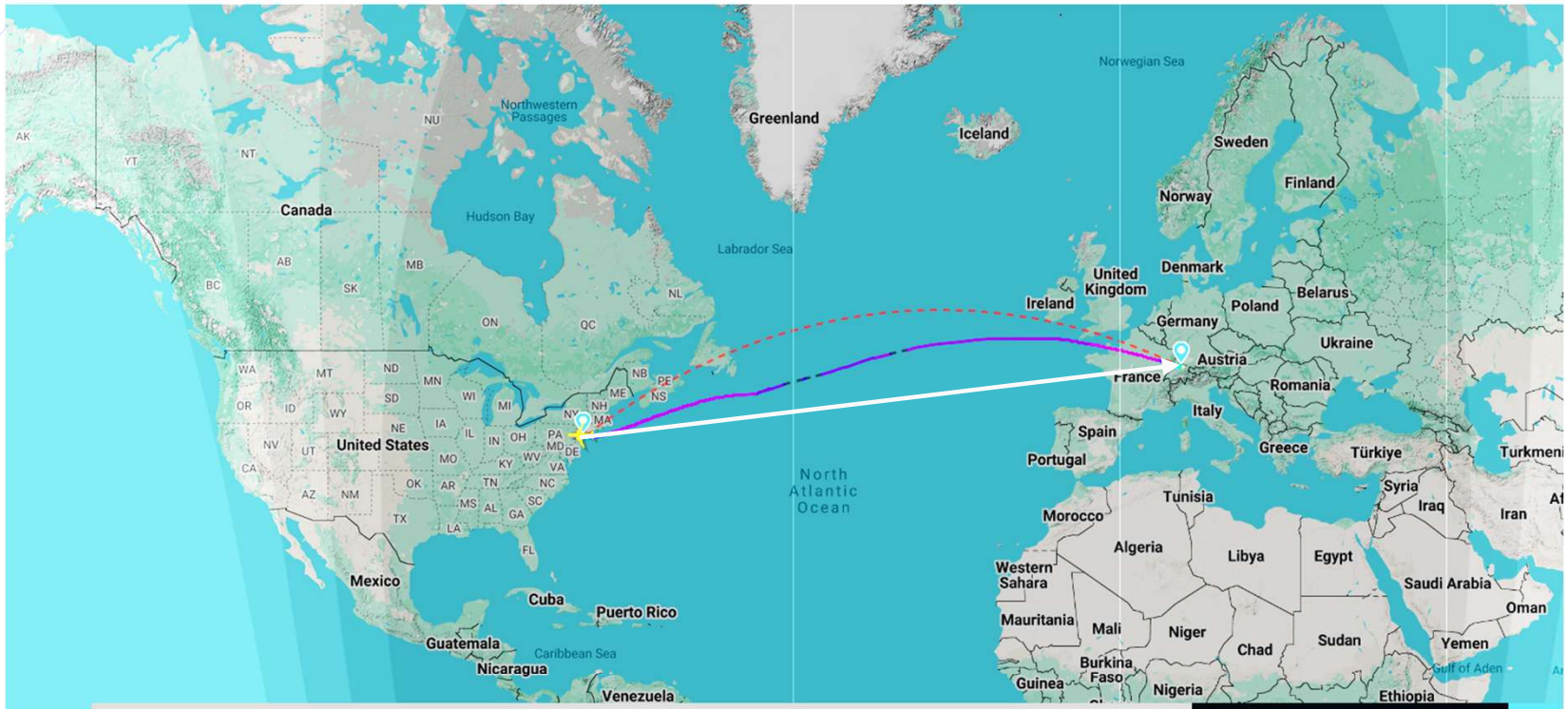


https://dirkmueller8.com/Coordinates/CoordForm

flightradar24 LIVE AIR TRAFFIC

LOG IN

Flight tracker map > Aviation data > Flights > DL52



Playback of flight DL52 / DAL52

https://www.flightradar24.com/data/flights/dl52#3e5abd5d

GREAT CIRCLE DISTANCE
6,326 KM

AVERAGE FLIGHT TIME
6:53

ACTUAL FLIGHT TIME
6:53

AVERAGE ARRIVAL DELAY
0:00

FROM
New York (JFK)

TO
Zurich (ZRH)

TIME
2:00 AM UTC

BAROMETRIC ALTITUDE
0 FT

GPS ALTITUDE

GROUND SPEED
0 KTS

TRUE AIRSPEED

VERTICAL SPEED

INDICATED AIRSPEED

TRACK
272°

SQUAWK

AIRCRAFT
Boeing 767-432(ER)

REGISTRATION
N839MH

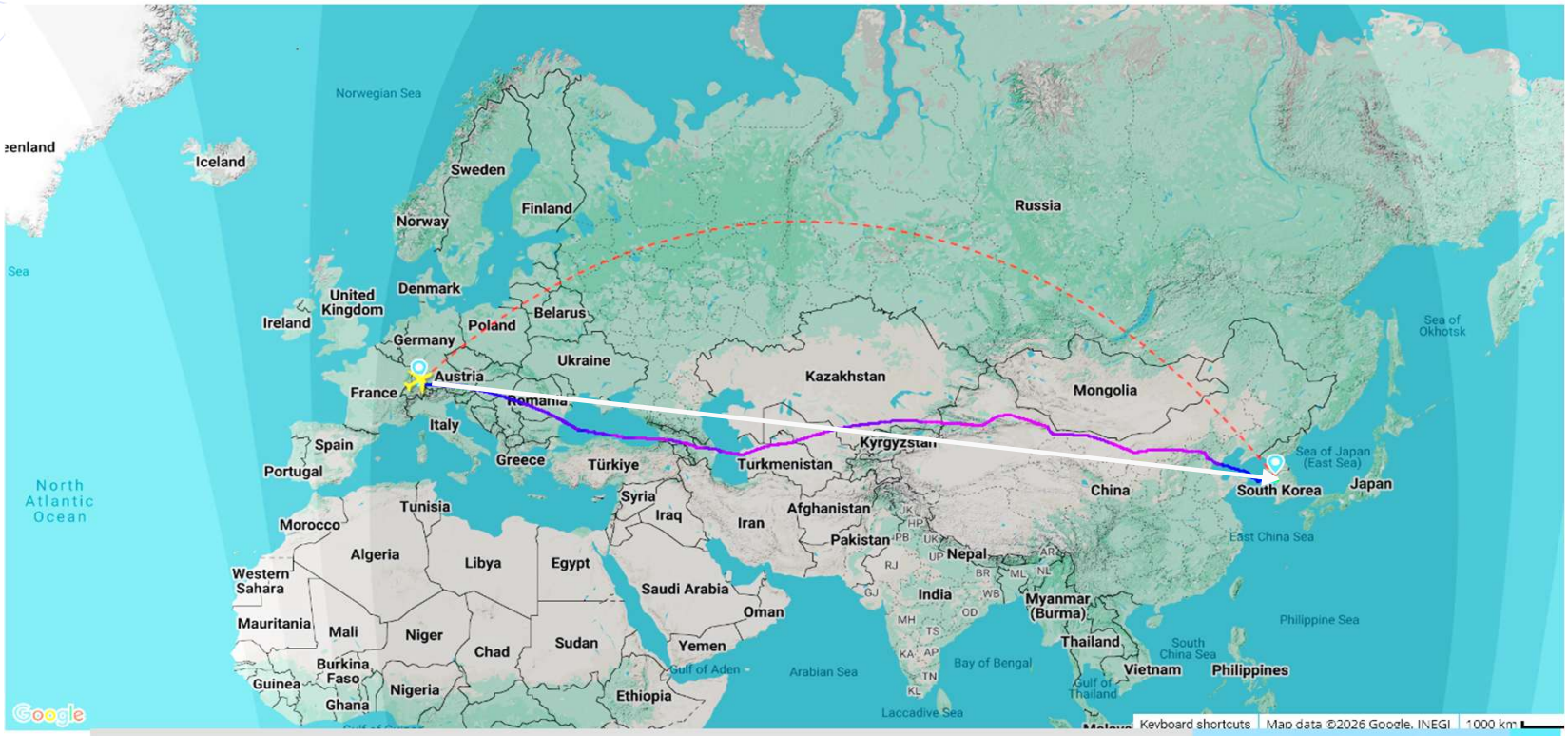
SERIAL NUMBER (MSN)

flightradar24

flightradar24 LIVE AIR TRAFFIC

LOG IN

Flight tracker map > Aviation data > Flights > KE568



Playback of flight KE568 / KAL568

GREAT CIRCLE DISTANCE
8,765 KM

AVERAGE FLIGHT TIME
11:05

ACTUAL FLIGHT TIME
10:38

AVERAGE ARRIVAL DELAY
0:00

FROM
Zurich (ZRH)

TO
Seoul (ICN)

TIME
7:06 PM UTC

BAROMETRIC ALTITUDE
0 FT

GPS ALTITUDE

GROUND SPEED
0 KTS

TRUE AIRSPEED

VERTICAL SPEED

INDICATED AIRSPEED

TRACK
64°

SQUAWK

AIRCRAFT
Boeing 777-FB5

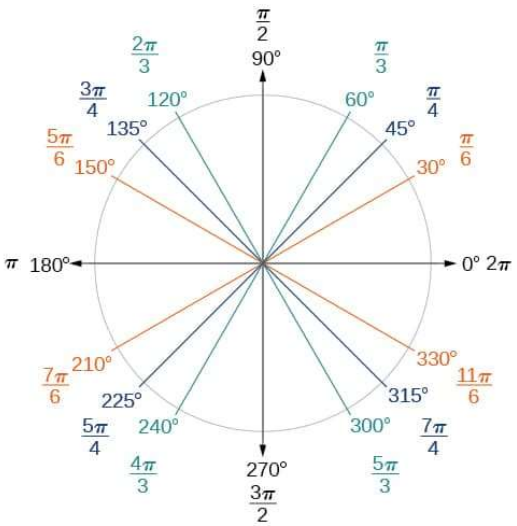
REGISTRATION
HL8043

SERIAL NUMBER (MSN)
-

flightradar24

Haversine Distance

```
When
Origin point = (ϕ₁ , λ₁)
Destination point = (ϕ₂ , λ₂)
(in radians)
```

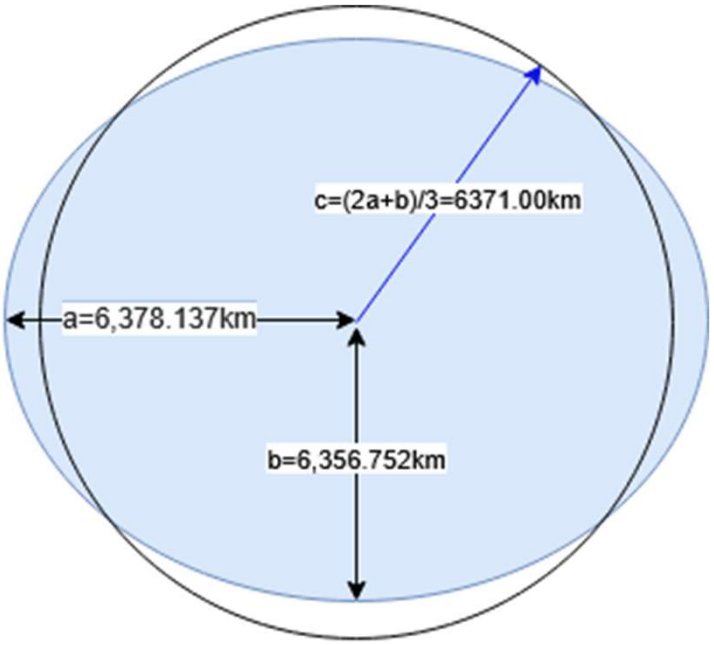



```
function haversine(ϕ₁, λ₁, ϕ₂, λ₂):

    # (if they are not radians) convert degrees to radians
    ϕ₁, ϕ₂ = ϕ₁, ϕ₂ × (π / 180)    # lat
    λ₁, λ₂ = λ₁, λ₂ × (π / 180)    # lon

    # Calculate differences of radians
    Δϕ = (ϕ₂ − ϕ₁)                 # difference in lat
    Δλ = (λ₂ − λ₁)                 # difference in long

    # Apply Haversine formula
    a  = sin²(Δϕ/2) + cos(ϕ₁) · cos(ϕ₂) · sin²(Δλ/2)
    c  = 2 · arctan2(√a, √(1 − a)) # central angle
    d  = 6371 · c                  # Earth mean radius in km

    return d                       # Haversine distance in km
```

In reality (newly added slide)

- In reality, the earth is an oblate spheroid, not a perfect sphere.
- ✓ Equatorial radius: 6,378 km
- ✓ Polar radius: 6,357 km
- ✓ → Difference: ~21 km, which is about 0.33% of the mean radius

- Haversine, therefore, may lead to some errors
- For more precise calculation, another method is needed, e.g. Vincenty's formula, which considers the oblate spheroid, i.e. ellipsoid



https://www.spatialanalysisonline.com/HTML/index.html?distance_operations.htm

Cartesian vs Haversine (brainstorm)

Let's measure the distance between

- Bern = (600500, 206750), Zurich = (682000, 252750)

If we use different measuring approaches, Cartesian coordinates (**Euclidian**, **Manhattan**) vs. Spherical geometry (**Haversine**), how much the differences can be?

What do you need to consider to calculate

- Cartesian coordinates? Spherical geometry?

How would you design the code?

I show you the example code…

Where to find all the codes and data

If you use Windows, in your File Explorer, copy paste the directory below

\\geofiles.d.uzh.ch\shared\course\geo877

If you use Mac, I am not familiar with how it works, but please get helped here:
https://www.geo.uzh.ch/en/services/it-services/geo-storage-spaces/paths.html

10” break

Join the Teams: **blyicls**
Sign up for three papers of your preference

How to measure the accuracy of (many) distances?

In earlier example, for a set of **two** points, we calculated the difference of two distance measures by subtracting the results between two measures, and this can be an indication of accuracy. But what if there are so many points, and we want to see the accuracy of all possible pairs' distances for two distance measures?

- Calculate the **mean** difference
- Calculate the **variance** of differences (i.e. range of difference)
- Calculate the **standard deviation** of differences
- Calculate the **maximum absolute error** and **minimum relative error** in distance pairs (i.e. the worst-case magnitude)

Mean (μ), variance(σ^2), and standard deviation (σ)

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

```
def variance(data):
    n = len(data)
    mean = sum(data) / n
    variance = sum((x-mean)**2 for x in data)/n
    return variance
```

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^2 - 2x_i\mu + \mu^2\right)$$

$$\sigma^2 = \frac{1}{n}\left(\sum x_i^2 - 2\mu(n\mu) + n\mu^2\right)$$

$$\sigma^2 = \frac{1}{n}\sum x_i^2 - \mu^2$$

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)^2$$

Think of efficiency

To compare two distance measures, we need to calculate every distance twice:
- once with Cartesian coordinates, and once with Haversine of WGS84

When the number of points are small, the efficiency (i.e. the computational load or duration time) will be miscellaneous

But once the data scales up, the efficiency – only a couple (mili)seconds, will matter a lot

Correctness and efficiency problems

The problem in earlier equation of variance is that:
e.g. when it handles two numbers (x and mean) that are often very close,
e.g. when the difference is small compared to the two numbers being handled (i.e. decimal points),
→ it can lose precision

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

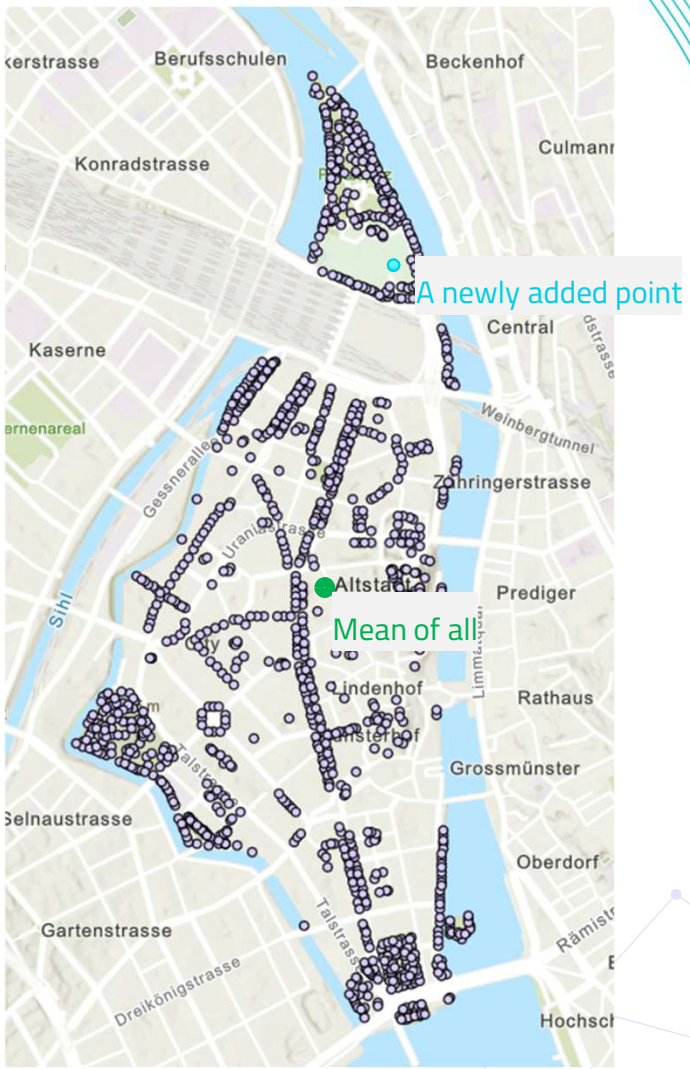
$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n}x_i\right)^2$$

Another more critical case is:
e.g. when the dataset is not complete from the beginning but continuously flows into, and thus the mean is constantly changing (i.e. incremental calculation)
→ the mean needs to be calculated again whenever the new value is added

Welford' algorithm

The idea of Welford's algorithm (1962): Instead of recalculating everything every time anew (esp. mean value), let's updates the **mean** and the **sum of squared deviations** from older values, so that it can keep previous values

- It avoids the precision issue
- It requires only one pass through the data, i.e., the computing process is much efficient and less resource-consuming (imagine the times of 1962)



https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance

Welford' algorithm

The idea of Welford's algorithm (1962):
How to update the **mean** and the **sum of squared deviations** from older values?

```
M_1 = x_1        # initial mean (M) is the same as first data read
S_1 = 0          # S stands for sq.dist from mean

for 2 ≤ k ≤ n {
    M_k = M_{k-1} + (x_k − M_{k-1})/k          # update mean
    S_k = S_{k-1} + (x_k − M_{k-1})*(x_k − M_k)    # update sum of square
}

S^2 = S_k / (k-1)                    # updated (sample) variance
```

How much a new value distant from the earlier mean

By definition,

$$\mu_n = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Split the sum into the first $n-1$ points and the new one:

$$\mu_n = \frac{1}{n}\left(\sum_{i=1}^{n-1} x_i + x_n\right)$$

The old mean is:

$$\mu_{n-1} = \frac{1}{n-1}\sum_{i=1}^{n-1} x_i$$

So:

$$\sum_{i=1}^{n-1} x_i = (n-1)\mu_{n-1}$$

Substitute:

$$\mu_n = \frac{(n-1)\mu_{n-1} + x_n}{n}$$

Rewrite:

$$\mu_n = \frac{n\mu_{n-1} - \mu_{n-1} + x_n}{n}$$
$$= \mu_{n-1} + \frac{x_n - \mu_{n-1}}{n}$$

https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance

Practical exercise:

- Brainstorm and design the code to calculate mean, variance, maximum absolute error and minimum relative error of two different distance measures (Euclidean vs Haversine) for a big set of points
- Calculate these metrics by each region
- (optional) Implement Welford's algorithm and test how efficient it can be

➢ We will discuss your solutions at the beginning of next class

Your group will be ready soon

Your group will be announced in Teams as soon as everyone's paper preference is ready

- You will find your teammates on Teams
- You will have access to your private channels on Teams shortly

Your first group work will be a paper review presentation, 10 minutes per group; the presentation should include:

1. What **research questions** were posed by the authors?
2. What **data** were used? How were the data **structured** and **organised**?
3. What **tasks** were required to carry out the analysis?
4. If you **replicate** this study in your own way, how will you define research question(s), data source(s), and algorithm(s)?